

Week 1: Requirement Analysis and Patient Database Module

1. Requirement Analysis

Understanding the Skincare Clinic Management System

A Skincare Clinic Management System is a database application designed to manage the daily activities of a skincare clinic. The system helps maintain patient records, appointments, treatments, skincare products, billing information, and staff details. A well-structured database ensures accurate record keeping and smooth clinic operations.

The Process Flow

- **Patient Registration:** Patients register by providing their personal information and medical details.
- **Appointment Booking:** Patients schedule appointments with dermatologists or skincare specialists.
- **Treatment Consultation:** The doctor examines the patient and recommends suitable treatments.
- **Treatment Management:** Treatment details and progress are recorded in the system.
- **Billing and Payment:** Bills are generated based on treatments and products used, and payments are recorded.
- **Product Management:** Skincare products used or sold are tracked through inventory records.
- **Follow-Up and Reports:** Patient treatment history and clinic reports are maintained for future reference.

Entities

Entity	Description	Key Attributes	Key Relationships
Patient	Stores information about patients receiving skincare services.	Patient_id (PK), first_name, last_name, age, gender, phone, email, address	One-to-Many with Appointment and Billing

Entity	Description	Key Attributes	Key Relationships
Appointment	Maintains appointment details between patients and doctors.	Appointment_id (PK), patient_id (FK), doctor_id (FK), appointment_date, appointment_time, status	Many-to-One with Patient and Doctor
Treatment	Stores information about skincare treatments provided to patients.	Treatment_id (PK), patient_id (FK), treatment_name, treatment_date, cost, remarks	Many-to-One with Patient
Billing	Maintains payment and billing details for treatments.	Bill_id (PK), patient_id (FK), treatment_id (FK), amount, payment_method, payment_date	Many-to-One with Patient and Treatment
Product	Stores details of skincare products available in the clinic.	Product_id (PK), product_name, category, price, stock_quantity, expiry_date	One-to-Many with Inventory
Staff	Maintains information about doctors and clinic employees.	Staff_id (PK), name, designation, phone, email, salary	One-to-Many with Appointment
Inventory	Tracks stock details of skincare products.	Inventory_id (PK), product_id (FK), stock_added, stock_used, available_stock	Many-to-One with Product

Patient Table Creation

The **Patient** table is created to store the personal and contact information of patients visiting the skincare clinic. Each patient is assigned a unique Patient ID which serves as the primary key for identification.

```
SQL> CREATE TABLE Patient ( patient_id NUMBER(5) PRIMARY KEY, first_name VARCHAR2(30) NOT NULL, last_name VARCHAR2(30), age NUMBER(3), gender VARCHAR2(10), phone VARCHAR2(15) UNIQUE, email VARCHAR2(50) UNIQUE, address VARCHAR2(100));
Table created.
```

Describe Patient Table

After creating the Patient table, the **DESCRIBE** command is used to verify the table structure. It displays the column names, data types, constraints, and other details of the table.

```
SQL> DESC patient
Name                               Null?    Type
-----
PATIENT_ID                        NOT NULL NUMBER(5)
FIRST_NAME                        NOT NULL VARCHAR2(30)
LAST_NAME                        VARCHAR2(30)
AGE                               NUMBER(3)
GENDER                           VARCHAR2(10)
PHONE                            VARCHAR2(15)
EMAIL                            VARCHAR2(50)
ADDRESS                           VARCHAR2(100)
```

Insert Values

The **INSERT** command is used to add patient information into the Patient table. Multiple patient records can be inserted into the database as required.

```
SQL> INSERT INTO Patient VALUES (101, 'Ananya', 'Sharma', 25, 'Female', '9876543210', 'ananya@gmail.com', 'Chennai');
1 row created.

SQL> INSERT INTO Patient VALUES (102, 'Priya', 'Rao', 30, 'Female', '9876543211', 'priya@gmail.com', 'Tambaram');
1 row created.

SQL> INSERT INTO Patient VALUES (103, 'Karthik', 'Kumar', 28, 'Male', '9876543212', 'karthik@gmail.com', 'Velachery');
1 row created.
```

Display Patient Records

The stored patient information can be viewed using the **SELECT** statement. This helps clinic staff retrieve patient details whenever needed.

```
SQL> SELECT * FROM Patient;

PATIENT_ID FIRST_NAME LAST_NAME
-----
          AGE GENDER  PHONE
-----
EMAIL
-----
ADDRESS
-----
          101 Ananya      Sharma
          25 Female    9876543210
ananya@gmail.com
Chennai

PATIENT_ID FIRST_NAME LAST_NAME
-----
          AGE GENDER  PHONE
-----
EMAIL
-----
ADDRESS
-----
          102 Priya      Rao
          30 Female    9876543211
priya@gmail.com
Tambaram
```

ADDRESS				
102	Priya			Rao
30	Female	9876543211		
priya@gmail.com				
Tambaram				
PATIENT_ID	FIRST_NAME		LAST_NAME	
AGE	GENDER	PHONE		
EMAIL				
ADDRESS				
103	Karthik			Kumar
28	Male	9876543212		
karthik@gmail.com				
Velachery				

Update

Existing patient information can be modified using the **UPDATE** statement. The **WHERE** clause is used to identify the specific patient record that needs to be updated.

```
SQL> UPDATE Patient SET phone = '9999999999' WHERE patient_id = 101;

1 row updated.
```

Conclusion

The Skincare Clinic Management System provides an efficient method for managing patient records, appointments, treatments, billing, product inventory, and staff information. The database helps improve accuracy, reduce manual work, and ensure smooth clinic operations. It enhances patient service while maintaining organized records for future reference.

THIRUVARASI. S

B.Sc CS WITH AI [S-1]

E25AI146